

Lab Assignment No.	10
Title	Create and Art with Neural style transfer on given image using deep learning.
Roll No.	
Class	BE AI & DS
Date Of Completion	
Subject	Computer Laboratory III[417534]
Assessment Marks	
Assessor's Sign	

Assignment No. 10

Title: Art Creation Using Neural Style Transfer with Deep Learning

Problem Statement: Neural Style Transfer (NST) is a deep learning technique used to combine the content of one image with the artistic style of another. This practical introduces the concept of NST and demonstrates how it can be implemented using a pre-trained deep learning model.

Prerequisites:

- Basic understanding of Convolutional Neural Networks (CNNs)
- Knowledge of Python programming
- Familiarity with deep learning frameworks like **TensorFlow** or **PyTorch**
- Understanding of the concept of **content** and **style** in images

Software Requirements:

- Python 3.x
- TensorFlow or PyTorch (for implementing Neural Style Transfer)
- Jupyter Notebook (or any Python IDE)
- Libraries: NumPy, Matplotlib, Keras, OpenCV, etc.

Hardware Requirements:

- A system with at least 8GB of RAM
- A Graphics Processing Unit (GPU) is recommended for faster computation (optional)
- Disk space for image datasets

Theory:

Neural Style Transfer (NST) is a technique in deep learning that allows the combination of the content of one image with the artistic style of another. The aim of NST is to create a new image that retains the structure and objects (content) of the source image, while adopting the visual appearance, color patterns, textures, and artistic style of the style image.

The underlying concept of NST can be explained through two primary concepts: **content** and **style**.

1. **Content:**

Content refers to the key structures and objects present in an image. In the context of NST, the content of an image is typically related to the arrangement of objects and their spatial relationships. For example, a portrait may contain content such as the shape of the face, the location of eyes, nose, and mouth, and other objects present in the scene.

2. **Style:**

Style refers to the visual appearance, textures, colors, and patterns found in an image. The style can be defined by elements like color schemes, textures, brush strokes, and patterns. In art, style often characterizes the work of specific artists (e.g., Van Gogh's swirling brushstrokes or Picasso's abstract shapes).

How Neural Style Transfer Works:

NST uses deep convolutional neural networks (CNNs) to separate the content and style of an image. A pre-trained network, such as **VGG-19**, is employed to extract deep features from both the content and style images.

1. **ContentLoss:**

To retain the content of the original image, the difference between the content features of the generated image and the original content image is measured. This difference is referred to as **content loss**, which is minimized during the optimization process.

2. **StyleLoss:**

To capture the style of the artistic image, a measure of the difference between the style features of the generated image and the style image is calculated. This difference is quantified using the **Gram matrix**, which represents the correlations between feature maps in the neural network. The **style loss** measures how well the generated image matches the style of the style image.

3. **GramMatrix:**

The Gram matrix is a method to capture the **correlation** between different channels (feature maps) of the image. It is computed by multiplying the feature map by its transpose. This matrix captures texture patterns, which are crucial for replicating an image's artistic style.

Loss Function:

The optimization in NST aims to minimize the **total loss**, which is a combination of the content and style losses. This is done iteratively by adjusting the pixels of the generated image.

- **Content Loss** is calculated as the difference between the content representation of the content image and the generated image.
- **Style Loss** is calculated as the difference between the style representation of the style image and the generated image, using the Gram matrix.
- The total loss function combines these two losses with respective weights, allowing the optimization process to balance content retention and style reproduction.

Optimization:

The optimization process uses **gradient descent** to minimize the total loss. By iteratively adjusting the pixels of the generated image, the model gradually creates an image that has the content of the original image and the style of the style image. The process typically uses an optimization algorithm such as **Adam** or **L-BFGS**.

Applications of Neural Style Transfer:**1. ArtisticRendering:**

NST can be used to generate artistic representations of photographs by applying the style of famous artists (e.g., Van Gogh, Picasso) to a regular image. This has widespread applications in digital art and design.

2. PhotoEnhancement:

NST allows for the transformation of everyday photographs into visually engaging works of art by transferring the style of selected artworks onto the images.

3. CreativeIndustries:

Artists and designers leverage NST to explore new creative possibilities by combining different art styles with real-world photographs, offering novel ways to express creativity.

Source Code –

```
# Import Required Libraries

import tensorflow as tf

import numpy as np

import matplotlib.pyplot as plt

from tensorflow.keras.preprocessing

import image from tensorflow.keras.applications import vgg19

# Load and Preprocess Image

def load_and_process_img(img_path,target_size=(128, 128)):

    img = image.load_img(img_path, target_size=target_size)

    img = image.img_to_array(img)

    img = np.expand_dims(img, axis=0)

    img = vgg19.preprocess_input(img)

    return img

def deprocess_img(img):

    img = img.squeeze() img = img + [103.939, 116.779, 123.68] # Convert back to BGR

    img = np.clip(img, 0, 255).astype('uint8')

    return img

# Load the Content and Style Images

content_path = r"C:\Users\saira\Downloads\Content.jpg"

style_path = r"C:\Users\saira\Downloads\style.jpg"

content_img = load_and_process_img(content_path)

style_img = load_and_process_img(style_path)

# Load the VGG19 Model

model = vgg19.VGG19(weights='imagenet', include_top=False)
```

```
# Define the layers we will use for content and style
```

```
content_layer = 'block5_conv2' # Content layer
```

```
style_layers = ['block1_conv1', 'block2_conv1', 'block3_conv1', 'block4_conv1', 'block5_conv1']
```

```
# Define a Function to Get Feature Representations
```

```
def get_feature_representations(model, content_img, style_img):
```

```
content_output = model.get_layer(content_layer).output
```

```
style_outputs = [model.get_layer(layer).output for layer in style_layers]
```

```
model = tf.keras.models.Model(inputs=model.input, outputs=[content_output] + style_outputs)
```

```
content_features = model(content_img)
```

```
style_features = model(style_img)
```

```
content_feature = content_features[0]
```

```
style_features = style_features[1:]
```

```
return content_feature, style_features
```

```
# Compute the Losses (Content, Style, and Total Variation Loss)
```

```
def compute_content_loss(content, generated):
```

```
return tf.reduce_mean(tf.square(content - generated))
```

```
def compute_style_loss(style, generated):
```

```
gram_style = gram_matrix(style)
```

```
gram_generated = gram_matrix(generated)
```

```
return tf.reduce_mean(tf.square(gram_style - gram_generated))
```

```
def gram_matrix(x):
```

```
channels = int(x.shape[-1])
```

```
a = tf.reshape(x, [-1, channels])
```

```
gram = tf.matmul(a, a, transpose_a=True)
```

```
return gram
```

```
def compute_total_variation_loss(generated):  
    return tf.reduce_mean(tf.image.total_variation(generated))  
  
# Compute the Gradients and Perform Optimization  
  
def compute_loss(content_feature, style_features, generated_img):  
    generated_content_feature, generated_style_features = get_feature_representations(model,  
        generated_img, generated)  
  
    content_loss = compute_content_loss(content_feature, generated_content_feature)  
  
    style_loss = sum(compute_style_loss(style, generated) for style, generated in zip(style_features,  
        generated_style)  
  
    total_variation_loss = compute_total_variation_loss(generated_img)  
  
    total_loss = content_loss + 0.025 * style_loss + 1.0 * total_variation_loss  
  
    return total_loss  
  
def compute_gradients(content_feature, style_features, generated_img):  
    with tf.GradientTape() as  
    tape: tape.watch(generated_img)  
  
    loss = compute_loss(content_feature, style_features, generated_img)  
  
    grads = tape.gradient(loss, generated_img)  
  
    return grads  
  
# Initialize and Optimize the Generated Image  
  
generated_img = tf.Variable(content_img, dtype=tf.float32)  
  
content_feature, style_features = get_feature_representations(model, content_img, style_img)  
  
optimizer = tf.optimizers.Adam(learning_rate=0.01)  
  
iterations = 200  
  
for i in range(iterations):  
  
    grads = compute_gradients(content_feature, style_features, generated_img)  
    optimizer.apply_gradients([(grads, generated_img)])  
  
    if i % 100 == 0:
```

```
print(f"Iteration {i}")  
  
img = deprocess_img(generated_img.numpy())  
  
plt.imshow(img)  
  
plt.show()  
  
# Display the Final Image  
  
final_img = deprocess_img(generated_img.numpy())  
  
plt.imshow(final_img)  
  
plt.show()
```

Conclusion: Neural Style Transfer demonstrates the ability of deep learning models to perform creative tasks by extracting content and style features from images and recombining them. This practical not only deepens the understanding of CNNs but also highlights their potential in applications beyond traditional image classification, such as artistic generation and creative design.